DAT540 project assignment – group 1

Can machine learning be used to differentiate between patients with and without gallstone disease and what features contributes most?

Ashvini Krishnakumar
Even Eide
Azhar A. Ahmad
Bård Garathun
Henriette Schjølset

Universitetet i Stavanger

Data exploration and cleaning

```
ref = {}

bmi_calc = galle_set['Weight'] / (galle_set['Height'] / 100) ** 2
ref['BMI differs from weight/height² (> 1.5)'] = (bmi_calc - galle_set['Body Mass Index (BMI)']).abs() > 1.5


ldl_calc = (galle_set['Total Cholesterol (TC)']
            - galle_set['High Density Lipoprotein (HDL)']
            - galle_set['Triglyceride'] / 5)
valid_tg = galle_set['Triglyceride'] < 400
ref['LDL differs from Friedewald (> 30, TG < 400)'] = valid_tg & ((ldl_calc - galle_set['Low Density Lipoprotein (LDL)']).abs() > 30)


ref['GFR < 30 with creatinine < 1.0'] = (galle_set['Glomerular Filtration Rate (GFR)'] < 30) & (galle_set['Creatinine'] < 1.0)
```

```
if 'galle_raw' not in globals():
    galle_raw = galle_set.copy()      # keep the original
galle_clean = galle_raw.copy()


bad_bmi = ref['BMI differs from weight/height² (> 1.5)']
galle_clean.loc[bad_bmi, 'Body Mass Index (BMI)'] = (
    galle_raw['Weight'] / (galle_raw['Height'] / 100) ** 2).round(1)[bad_bmi]

bad_gfr = ref["GFR < 30 with creatinine < 1.0"]
gfr_col = "Glomerular Filtration Rate (GFR)"



imputer = SimpleImputer(strategy="median") # Create an imputer that uses the median to fill incorrect values.
imputer.fit(galle_clean.loc[~bad_gfr, [gfr_col]]) # Fit the imputer using only valid GFR values.

galle_clean.loc[bad_gfr, gfr_col] = imputer.statistics_[0]

display(galle_clean.loc[bad_gfr, [gfr_col]])

galle_flags = pd.DataFrame({
    'flag_dm_glucose': ref['DM = 0 but glucose >= 126'] | ref['DM = 1 but glucose <= 99'],
    'flag_friedewald': ref['LDL differs from Friedewald (> 30, TG < 400)'],
}).astype(int)
```

- Data quality issues
- Medical established reference values
- Statistical outliers
- Created flags
- Remove records

Preprocessing

- Outlier detection using IQR method
- Other plausible outliers retained
- Obesity value 1954 with median
- StandardScaler applied to continous features
- Validation set/unseen data
- Stratified train/test split 80/20

```
#Looking for outliers using IQR method
#first find the numerical features in the dataset with more than 5 different values
continous_features=[
    column for column in galle_set.select_dtypes(include='number').columns
    if galle_set[column].nunique() > 5
]
print ("Selected continous_features:")
for feature in continous_features:
    print(feature)
```

```
#check for outliers in continous features by using IQR method
outlier_results=[]
for column in continous_features:
    Q1 = galle_set[column].quantile(0.25)
    Q3 = galle_set[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = galle_set[(galle_set[column] < lower_bound) | (galle_set[column] > upper_bound)]

    outlier_results.append({
        'Feature': column,
        'Lower bound': round(lower_bound, 2),
        'Upper bound': round(upper_bound, 2),
        'Number of outliers': len(outliers),
        'Outlier values': sorted(outliers[column].tolist())
    })

#Want to sort from most to fewest outliers
outlier_summary=pd.DataFrame(outlier_results)
outlier_summary=outlier_summary.sort_values(
    by='Number of outliers',
    ascending=False
)
display(outlier_summary)
```

```
#Investigation of the suspicious outliers
suspicious_rows=galle_set[
    (galle_set['Obesity (%)']>100) |
    (galle_set['High Density Lipoprotein (HDL)'] > 250) |
    (galle_set['Muscle Mass (MM)'] < 8) |
    (galle_set['Body Protein Content (Protein) (%)'] < 10)
]
```

Feature selection

- Correlation-based feature selection
- Identified top 10 features by absolute correlation
- Strong correlations between predictors examined

```
#Feature selection based on correlation
correlation=galle_set.corr(numeric_only=True)['Gallstone Status']

#Sort by absolute correlation. Want to find the most correlatedfeatures, ev
correlation=correlation.drop('Gallstone Status')
top_features=correlation.abs().sort_values(ascending=False).head(10)

print(top_features)
```

```
#Check correlation between top 10 features
selected_features=top_features.index
feature_correlation=galle_set[selected_features].corr()

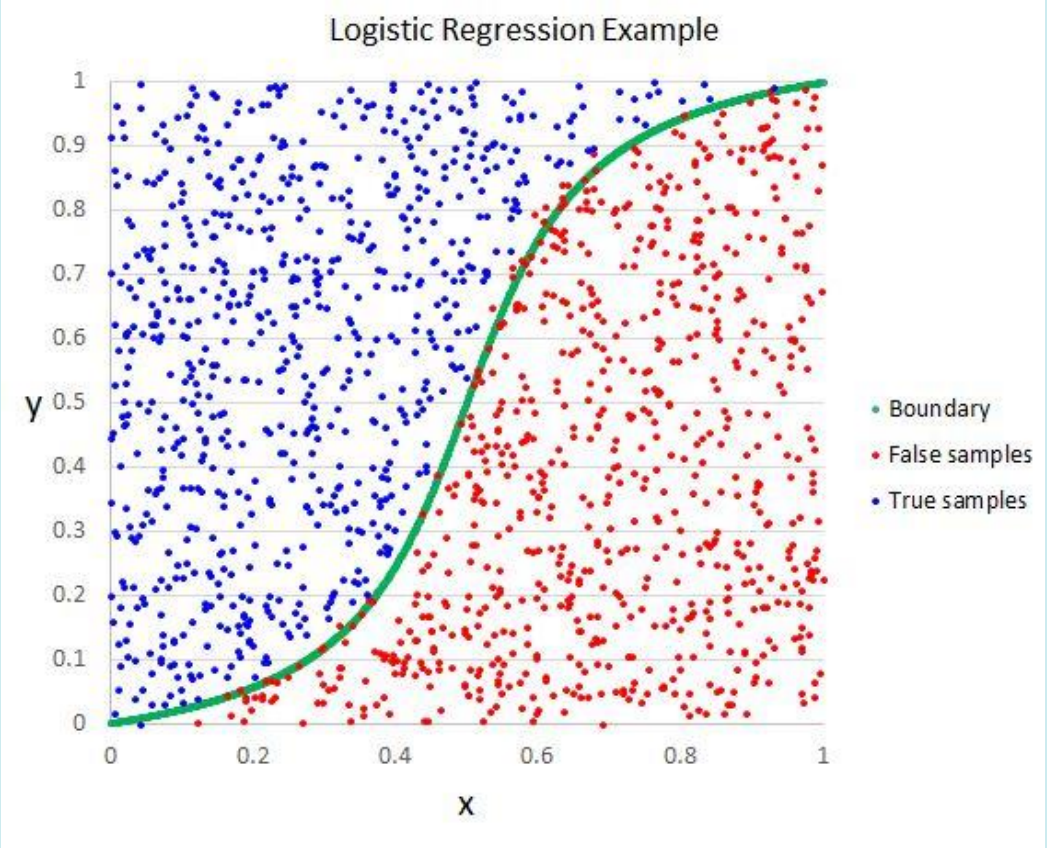
display(feature_correlation.round(2))
```

```
#Early feature selection basen on the correlation analysis, which the feature we want to continue with
SELECTED_FEATURES=[
    'Vitamin D',
    'C-Reactive Protein (CRP)',
    'Total Body Fat Ratio (TBFR) (%)',
    'Bone Mass (BM)',
    'Hemoglobin (HGB)',
    'Extracellular Fluid/Total Body Water (ECF/TBW)',
    'Hyperlipidemia'
]

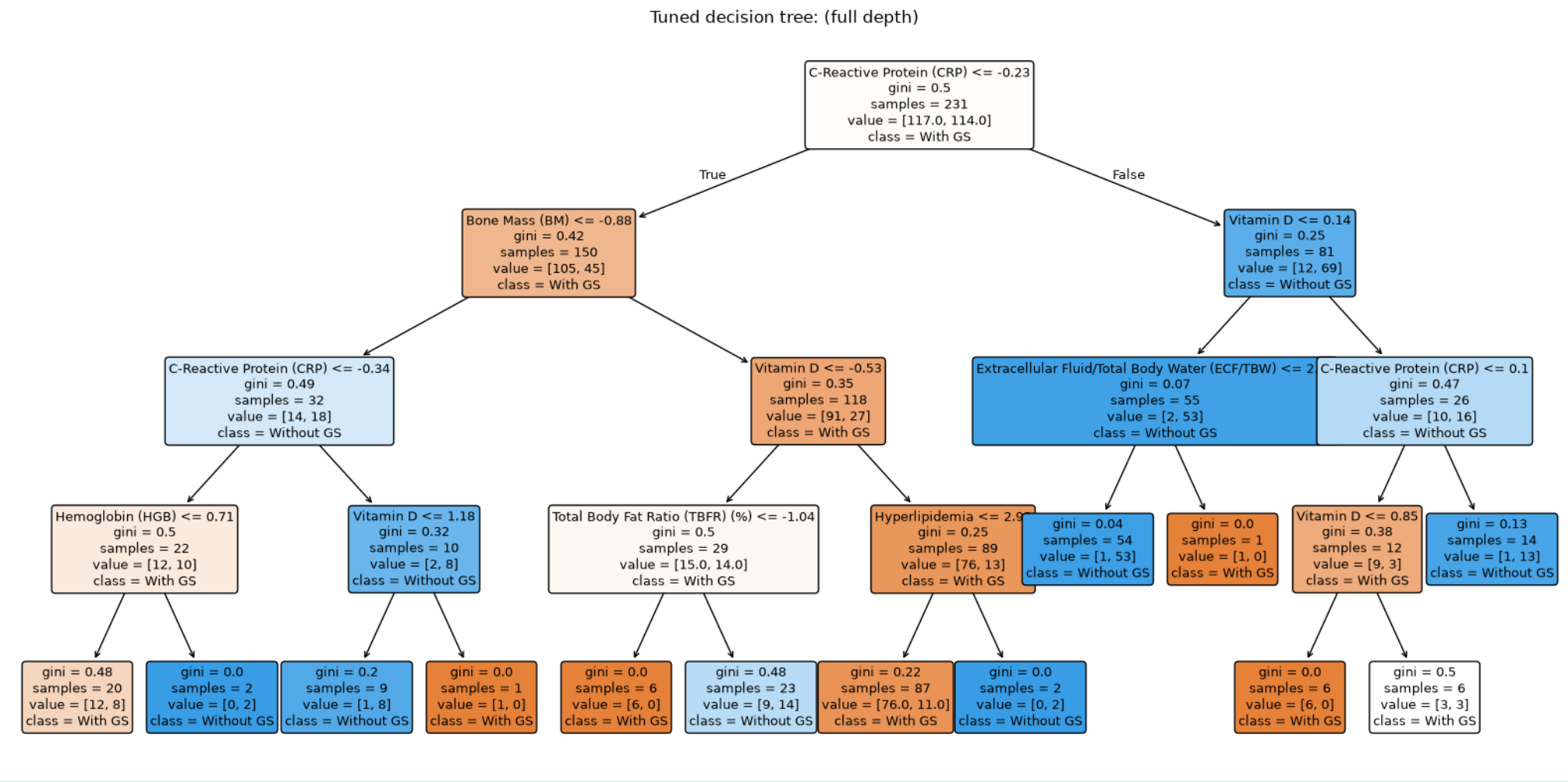
print("Number of selected features:", len(SELECTED_FEATURES))
print(SELECTED_FEATURES)
```

Model implementation

- Logistic regression



Decision Tree - Concept



Decision Tree – Parameter/Optimizing

```python
# Selecting the same features and order for every decision tree step
X_train_dt = X_train
X_test_dt = X_test
X_val_dt = validation_set_df.loc[:, SELECTED_FEATURES].copy()
y_val_dt = validation_set_df[TARGET]

# Defining features and target variable for decision tree models
dt_class_names = ["With gallstones (0)", "Without gallstones (1)"]
dt_cv = StratifiedKFold(n_splits=CV_SPLITS, shuffle=True, random_state=RANDOM_STATE)

# Defining models
dt_models = {
    "Initial tree": tree.DecisionTreeClassifier(random_state=RANDOM_STATE),
    "Depth 3 tree": tree.DecisionTreeClassifier(max_depth=3, random_state=RANDOM_STATE),
}

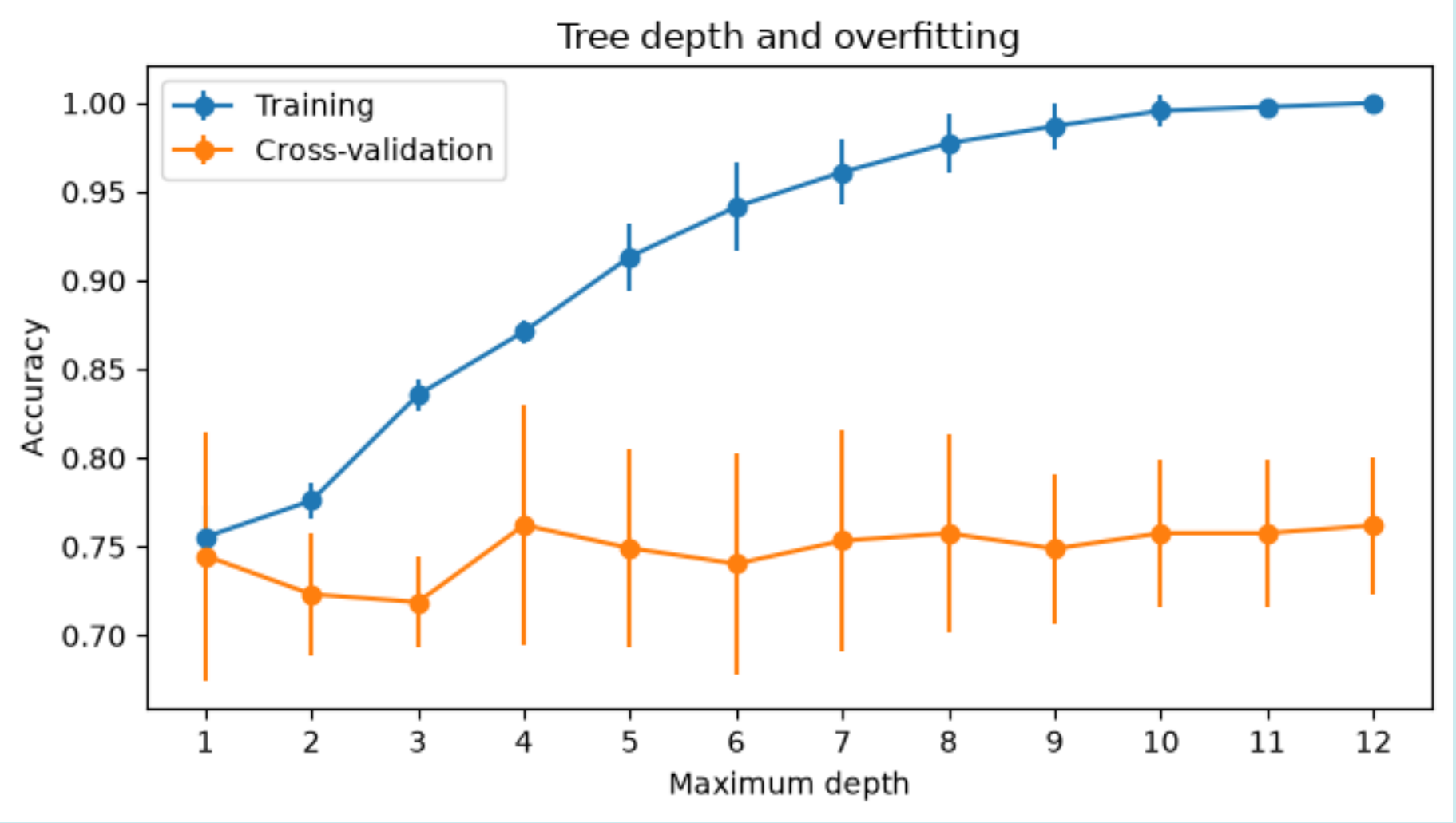
# Evaluating models using cross-validation
dt_cv_scores = {
    name: cross_val_score(model, X_train_dt, y_train, cv=dt_cv, scoring="accuracy").mean()
    for name, model in dt_models.items()
}
```

```python
## Setting up hyperparameter grid
dt_param_grid = {
    "criterion": ["gini", "entropy"],
    "max_depth": [2, 3, 4, 5, 6, None],
    "min_samples_split": [2, 10, 20],
    "min_samples_leaf": [1, 3, 5, 10],
    "ccp_alpha": [0.0, 0.005, 0.02],
}

# Performing grid search
dt_search = GridSearchCV(
    dt_models["Initial tree"],
    dt_param_grid,
    cv=dt_cv,
    scoring="accuracy"
)

# Fitting grid search to training data
dt_search.fit(X_train_dt, y_train)
dt_models["Tuned tree"] = dt_search.best_estimator_
dt_cv_scores["Tuned tree"] = np.float64(dt_search.best_score_)
```

Decision Tree – Evaluating/overfitting



Random Forest

- Training

```
Train accuracy: 0.952
Test accuracy: 0.862
Overfit gap: 0.090
```

```python
def train_model(self, **params):
    """
    Trains the model on the split data
    """
    defaults = dict(n_estimators = 1000, max_features = 0.3, max_depth = 4,
                    min_samples_leaf = 3, criterion = "entropy",
                    oob_score = True, n_jobs = -1, random_state  = self.random_state,
                    bootstrap = True, warm_start = False, class_weight = "balanced_subsample")
    defaults.update(params) # Update defaults with new parameters

    self.model = RandomForestClassifier(**defaults)
    self.model.fit(self.X_train, self.y_train)

    # Get predicted class probabilities for the test set
    y_pred_prob = self.model.predict_proba(self.X_test)[:, 1]
    predictions = self.model.predict(self.X_test)
    train_score = self.model.score(self.X_train, self.y_train)
    test_score = self.model.score(self.X_test, self.y_test)

    print(f"OOB accuracy:  {self.model.oob_score_:.3f}")
    print(f"Test accuracy: {accuracy_score(self.y_test, predictions):.3f}")
    print(f"ROC AUC Score: {roc_auc_score(self.y_test, y_pred_prob):.3f}\n")
    print("Classification report")
    print(classification_report(self.y_test, predictions))

    print(f"Train accuracy: {train_score:.3f}")
    print(f"Test accuracy: {test_score:.3f}")
    print(f"Overfit gap: {train_score - test_score:.3f}")

    cm = confusion_matrix(self.y_test, predictions, labels=self.model.classes_)
```

```python
rf = RandomForrest(random_state = RANDOM_STATE, test_size = TEST_SIZE, target = TARGET, features = SELECTED_FEATURES,
                   testing_set_df = testing_set_df, X_train = X_train, X_test = X_test, y_train = y_train, y_test = y_test)
                    # Initiate RandomForrest
rf.train_model(max_depth = None, max_features = "sqrt", min_samples_leaf = 3, n_estimators = 300) # Calling train_model with params
```

Random Forest

- Validation

```python
def evaluate(self, model):
    """
    Evaluate the trained model against unseen data
    """
    self.X_val = self.validation_set_df[self.features]
    pred = model.predict(self.X_val)
    y_pred_prob = model.predict_proba(self.X_val)[:, 1]
    print(f"Colums used for validating:\nColumns: {len(self.X_val.columns)}\n{list(self.X_val)}\n")

    out = self.X_val[self.features].copy()
    out["P(class 1)"] = y_pred_prob.round(2)
    out["Predicted"]  = pred
    out["Actual"] = self.validation_set_df[self.target].to_numpy()
    out["Correct"] = out["Predicted"] == out["Actual"]
    self.y_val = self.validation_set_df[self.target]

    # plot
    print(f"Unseen accuracy: {accuracy_score(self.y_val, pred):.3f}  (n = {len(self.y_val)})")
    print(f"Unseen ROC-AUC:  {roc_auc_score(self.y_val, y_pred_prob):.3f}")
    print(classification_report(self.y_val, pred))
    print(f"ROC AUC Score: {roc_auc_score(self.y_val, y_pred_prob):.3f}")
    fig, ax =  plt.subplots(1, 2, figsize=(12, 5))
    ConfusionMatrixDisplay.from_predictions(self.y_val, pred, ax = ax[0], cmap = "Blues")
    ax[0].set_title("Confusion matrix (unseen data)")
    RocCurveDisplay.from_predictions(self.y_val, y_pred_prob, ax = ax[1])
    ax[1].plot([0, 1], [0, 1], "--", color = "grey", label = "Random guess")
    ax[1].legend()
    ax[1].set_title("ROC curve (unseen data)")
    plt.tight_layout()
    plt.show()

    display(out.head(30))
```

```python
rfv = RandomForrestValidation(target = TARGET, features = SELECTED_FEATURES)
rfv.evaluate(rf.model)
```

Model training

- How the models were trained:
  - Pre-processed data-set
  - All models based on same traning/testing split

Evaluation and conclusion

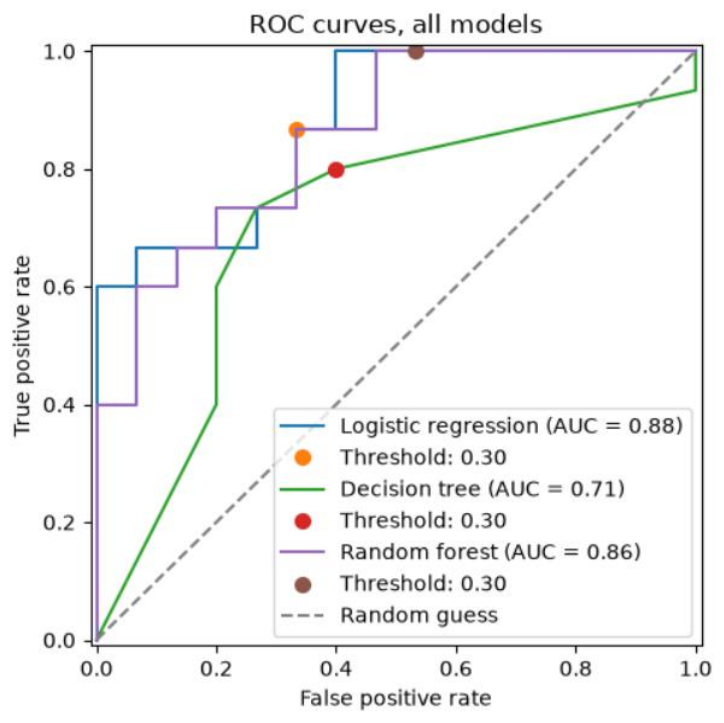
- Training performance
  - Training data (58)
  - Fit to training data

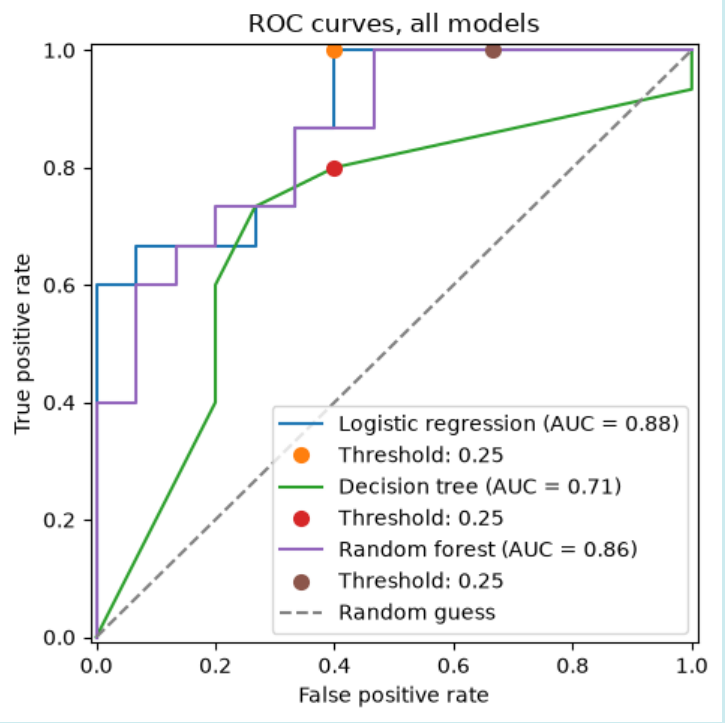
Model	Test accuracy	Precision (macro)	Recall (macro)	F1-score (macro)	Support
Logistic regression	0.845	0.855	0.845	0.844	58
Decision tree	0.759	0.760	0.759	0.758	58
Random forest	0.862	0.864	0.862	0.862	58

- Validation performance
  - Unseen data (30)
  - Tested on new patients

Model	Accuracy	Precision (macro)	Recall (macro)	F1-score (macro)	Support
Logistic regression	0.767	0.778	0.767	0.764	30
Decision tree	0.733	0.733	0.733	0.733	30
Random forest	0.733	0.733	0.733	0.733	30

Evaluation and conclusion





	Vitamin D	C-Reactive Protein (CRP)	Total Body Fat Ratio (TBFR) (%)	Bone Mass (BM)	Hemoglobin (HGB)	Extracellular Fluid/Total Body Water (ECF/TBW)	Hyperlipidemia	P(class 1)	Predicted	Actual	Correct
280	0.030916	3.443250	2.839427	0.389196	-2.087041	0.854253	-0.163299	0.87	1	1	True
60	0.405284	-0.387877	-0.967793	1.171824	0.684847	-0.671859	-0.163299	0.07	0	0	True
297	-0.635766	-0.387877	1.833092	-0.197774	-0.758844	1.464697	-0.163299	0.31	0	1	False
69	-1.281936	-0.192838	-0.633174	0.193539	1.608810	0.549030	-0.163299	0.70	1	0	False
222	0.287333	1.237450	1.631081	-1.176059	-1.336321	-1.465437	-0.163299	0.94	1	1	True
204	0.584776	-0.009408	-0.013509	1.563138	-0.152494	-0.671859	-0.163299	0.43	0	1	False

Conclusion

Machine learning can predict if a patient has gallstone disease and identify most cases with further tuning